Escape Sequence and Format Specifiers

In Java, you can format strings using **escape sequences** and **format specifiers** within methods like `System.out.printf()` or `String.format()`. This allows for more control over how text is displayed, such as inserting new lines, formatting numbers, or aligning text.

Escape Sequences

1. `\n` – Newline
   - Inserts a line break (moves the cursor to the next line).
   - **Example**:

```
System.out.println("Hello\nWorld");
// Output:
// Hello
// World
```

2. `\t` – Tab
   - Inserts a horizontal tab.
   - **Example**:

```
System.out.println("Name\tAge");
System.out.println("John\t30");
System.out.println("Alice\t25");
// Output:
// Name    Age
// John    30
// Alice   25
```

3. `\\` – Backslash
   - Inserts a backslash (`\`).
   - **Example**:

```
System.out.println("This is a backslash: \\");
// Output: This is a backslash: \
```

4. `\'` – Single Quote
   - Inserts a single quote (`'`).
   - **Example**:

```
System.out.println("It's a great day!");
// Output: It's a great day!
```

5. `\"` – Double Quote
   - Inserts a double quote (`"`).
   - **Example**:

```
System.out.println("She said, \"Hello!\"");
// Output: She said, "Hello!"
```

Format Specifiers

When you want to format strings, numbers, and other data types in a specific way, you can use **format specifiers** like `%d`, `%n`, `%s`, and others. These specifiers are commonly used with methods such as `System.out.printf()` and `String.format()`.

1. `%d` – Integer (Decimal)

- Used to format integers (both positive and negative).
- **Example**:

```
int age = 25;
System.out.printf("Age: %d\n", age);
// Output: Age: 25
```

2. `%s` – String

- Used to format strings. It's commonly used to insert a string value into a format.
- **Example**:

```
String name = "John";
System.out.printf("Hello, %s!\n", name);
// Output: Hello, John!
```

3. `%n` – Newline

- Inserts a platform-specific line separator (used for portability, as `\n` may not work the same across different operating systems).
- **Example**:

```
System.out.printf("This is line 1%nThis is line 2%n");
// Output:
// This is line 1
// This is line 2
```

4. `%f` – Floating-point Numbers

- Used to format floating-point numbers (e.g., `float` or `double`).
- **Example**:

```
double price = 99.99;
System.out.printf("Price: %.2f\n", price); // Rounds to 2 decimal
places
// Output: Price: 99.99
```

5. `%x` – Hexadecimal (Lowercase)

- Used to format numbers as hexadecimal.
- **Example**:

```
int number = 255;
System.out.printf("Hexadecimal: %x\n", number);
// Output: Hexadecimal: ff
```

6. `%o` – Octal

- Used to format numbers as octal.
- **Example**:

```
int number = 8;
System.out.printf("Octal: %o\n", number);
// Output: Octal: 10
```

7. `%c` – Character

- Used to format a single character.
- **Example**:

```
char grade = 'A';
System.out.printf("Grade: %c\n", grade);
```

```
// Output: Grade: A
```

Putting It All Together

```
public class StringFormattingExample {
    public static void main(String[] args) {
        // Using escape sequences
        System.out.println("Hello\nJava!");
        System.out.println("Tab\tSeparated\tValues");

        // Using format specifiers
        int age = 25;
        String name = "John";
        double price = 99.99;

        System.out.printf("Name: %s%n", name);
        System.out.printf("Age: %d%n", age);
        System.out.printf("Price: %.2f%n", price);

        // Using newline with %n for portability
        System.out.printf("Hello,%s!%nThis is a new line.%n", name);

        // Hexadecimal format
        int number = 255;
        System.out.printf("Hexadecimal: %x%n", number);

        // Octal format
        System.out.printf("Octal: %o%n", number);
    }
}
```

Output:

```
Hello
Java!
Tab    Separated    Values
Name: John
Age: 25
Price: 99.99
Hello,John!
This is a new line.
```

```
Hexadecimal: ff
Octal: 377
```
